

Matplotlib, seaborn, plotly, altair lequel choisir ?



Un arbre de décision pour choisir la bonne librairie de dataviz selon le livrable, sans perdre une soirée à comparer des tutos.

Ces quatre librairies ne se valent pas selon le besoin. La question n'est pas « laquelle est la meilleure », mais « qu'est-ce que je dois produire, et pour qui ». Ce guide te donne les 4 librairies en 30 secondes, puis 8 situations concrètes avec la librairie recommandée, le pourquoi, et un snippet prêt à copier.

Les 4 librairies en 30 secondes.

Avant de choisir, sache ce que chacune fait bien et où elle te bloque. Le réflexe « je connais matplotlib, donc je l'utilise partout » coûte des heures sur un dashboard.

1. matplotlib

Fiche : Tu veux un contrôle total sur chaque pixel : police, épaisseur de trait, position exacte d'une annotation. C'est la librairie bas niveau sur laquelle seaborn est construite.

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
ax.bar(df["categorie"], df["valeur"])
ax.set_title("Ventes par categorie")
plt.savefig("ventes.png", dpi=150)
```

Force : aucune dépendance implicite, sortie stable en PNG ou PDF haute résolution, pensée pour l'impression.

Limite : verbeux. Chaque réglage (titre, légende, couleurs) se code à la main, sans interactivité native.

2. seaborn

Fiche : Tu veux un graphe statistique correct en deux lignes, directement depuis un DataFrame pandas, sans configurer les axes toi-même.

```
import seaborn as sns
sns.histplot(df, x="montant", hue="segment", bins=30)
```

Force : construite sur matplotlib, API pensée pour pandas (**histplot**, **boxplot**, **heatmap**, **pairplot**), styles par défaut déjà lisibles.

Limite : reste statique comme matplotlib. Toute personnalisation fine repasse par les objets matplotlib sous-jacents.

3. plotly

Fiche : Tu veux un graphe qu'on peut survoler, zoomer, filtrer, dans un navigateur ou un dashboard consulté au quotidien.

```
import plotly.express as px
fig = px.line(df, x="date", y="ca", color="region")
fig.show()
```

Force : interactif nativement (tooltip, zoom, clic sur légende), s'exporte en HTML autonome, s'intègre à Dash ou Streamlit.

Limite : fichier plus lourd, rendu PDF statique moins soigné qu'un matplotlib pensé pour l'impression.

4. altair

Fiche : Tu veux déclarer « quelles données, quel encodage » et laisser la librairie choisir le rendu, plutôt que de piloter chaque élément toi-même.

```
import altair as alt
alt.Chart(df).mark_line().encode(
    x="date:T", y="ca:Q", color="region:N")
```

Force : grammar of graphics basée sur Vega-Lite, code court et lisible, bien adaptée à un notebook reproductible et versionné.

Limite : par défaut, elle plafonne le nombre de lignes affichées sans configuration explicite, et son écosystème reste plus petit que celui de matplotlib.

Le repère en une ligne

Avant de coder, pose-toi une seule question : le graphe part dans un document figé, ou dans un écran qu'on manipule ?

matplotlib

Statique, contrôle total

seaborn

Statique, stats rapides

plotly

Interactif, web

altair

Déclaratif, reproductible

8 situations, la bonne librairie.

Ici, la question n'est plus « quelle librairie je connais » mais « qu'est-ce que ce livrable exige ». Le choix change le résultat, pas seulement le style.

5. Explorer vite une distribution

Décision : Tu reçois un fichier et tu veux voir la forme d'une colonne (asymétrie, valeurs extrêmes) en quelques secondes, pas produire un livrable soigné. Recommandation : **seaborn**, avec **histplot** ou **boxplot**.

```
sns.boxplot(df, x="segment", y="montant")
```

Pourquoi : une ligne suffit sur un DataFrame pandas, sans configurer les axes à la main.

Limite de l'alternative : en matplotlib pur, il faut calculer les bins et gérer l'axe soi-même, pour un résultat exploratoire qui sera jeté dans la minute.

6. Un graphe pour un rapport PDF statique

Décision : Le livrable final est un PDF ou un PNG intégré à un rapport, imprimé ou joint à un mail. Recommandation : **matplotlib**.

```
plt.savefig("rapport_fig1.png", dpi=200,
            bbox_inches="tight")
```

Pourquoi : export natif haute résolution, pas de dépendance JavaScript, rendu identique quel que soit le poste qui l'ouvre.

Limite de l'alternative : un export plotly en PDF perd l'interactivité et rend souvent moins net qu'un matplotlib pensé pour l'impression.

7. Un dashboard web interactif

Décision : Le graphe vit dans une app web ou un dashboard que l'utilisateur manipule lui-même. Recommandation : **plotly**.

```
fig = px.scatter(df, x="prix", y="qte",
                 color="categorie")
fig.show()
```

Pourquoi : zoom, déplacement, filtre par clic sur la légende, sans code JavaScript supplémentaire.

Limite de l'alternative : matplotlib et seaborn restent des images figées, aucune interaction possible côté navigateur.

8. Survoler pour voir les valeurs TOOLTIP

Décision : Le lecteur doit pouvoir passer la souris sur un point et lire la valeur exacte, sans cliquer nulle part. Recommandation : **plotly**, via **hover_data**.

```
px.scatter(df, x="date", y="ca",
           hover_data=["client", "montant"])
```

Pourquoi : le tooltip est natif, tu ajoutes juste les colonnes que tu veux voir apparaître au survol.

Limite de l'alternative : matplotlib peut simuler un tooltip avec la librairie externe **mplcursors**, une dépendance de plus pour un besoin que plotly couvre nativement.

① Le repère qui tranche vite

Une seule question suffit la plupart du temps.

Le graphe part dans un PDF ou s'imprime
matplotlib (ou seaborn si le calcul stat est fait pour toi)

Le graphe part dans un navigateur ou une app
plotly (interactif) ou altair (déclaratif, versionné)

9. Corrélations rapides

Décision : Tu veux repérer vite les variables qui bougent ensemble, sur un jeu de colonnes numériques. Recommandation : **seaborn**, heatmap sur la matrice de corrélation ou pairplot pour un premier tour.

```
sns.heatmap(df.corr(numeric_only=True),
            annot=True, cmap="Greens")
```

Pourquoi : **sns.heatmap** prend directement une matrice de corrélation et l'annote sans code supplémentaire.

Limite de l'alternative : en matplotlib pur, il faut construire la matrice de couleurs et les annotations texte à la main, pour un diagnostic qui doit rester rapide.

10. Un graphe reproductible et lisible en code

Décision : Le graphe doit être versionné, relu en code review, et généré de façon identique à chaque exécution. Recommandation : **altair**.

```
alt.Chart(df).mark_bar().encode(
    x="mois:O", y="ca:Q")
```

Pourquoi : la syntaxe déclarative encode/mark décrit exactement ce qui est affiché, sans état mutable ni objet ax à faire circuler entre fonctions.

Limite de l'alternative : un matplotlib impératif, avec plusieurs appels successifs à **ax.set_**, devient vite difficile à relire en diff de code review.

11. Personnaliser chaque pixel pour une publication

Décision : Le graphe part dans un article, un mémoire ou une publication qui impose une charte graphique précise. Recommandation : **matplotlib**.

```
ax.annotate("pic de mars", xy=(3, 120),
            xytext=(4, 140),
            arrowprops=dict(arrowstyle="->"))
```

Pourquoi : chaque élément (police, épaisseur de trait, position d'une annotation) est un objet accessible et modifiable individuellement.

Limite de l'alternative : seaborn et plotly imposent un style par défaut plus difficile à casser finement pour respecter une charte externe stricte.

12. Intégrer dans Streamlit

Décision : Le graphe doit vivre dans une app Streamlit que l'équipe métier utilise au quotidien. Recommandation : **plotly** ou **altair**, selon l'interactivité voulue.

```
st.plotly_chart(fig, use_container_width=True)
# ou : st.altair_chart(chart)
```

Pourquoi : **st.plotly_chart** et **st.altair_chart** acceptent directement l'objet figure, sans conversion.

Limite de l'alternative : **st.pyplot()** fonctionne aussi pour matplotlib ou seaborn, mais tu perds le zoom et le tooltip que Streamlit peut afficher nativement avec plotly.

L'arbre de décision en une table

Si tu hésites encore après ce guide, pose ces quatre questions dans l'ordre.

Le livrable est un PDF ou une image imprimée?

Tu explores vite un DataFrame, sans livrable soigné?

Il faut du survol ou du zoom dans un navigateur?

Le code doit rester déclaratif et versionnable?

matplotlib
seaborn
plotly
altair

4 librairies, 8 situations. Retiens surtout la question à te poser avant d'ouvrir ton éditeur : qu'est-ce que je dois produire, et pour qui. Le choix de la librairie découle du livrable, pas de tes habitudes.

Fait main, en solo.

Si ce guide t'a aidé, partage-le autour de toi : il peut débloquent un autre candidat. Et si tu veux que je traite un sujet précis, dis-le moi en commentaire : c'est là que je pioche mes prochains guides.

Dylan

